

Embedded software organization

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Recap: von Neumann architecture

- Instructions are fetched from ROM on the same bus as data fetched from RAM
- Software is an ordered sequence of very specific instructions
- The processor executes the software by retrieving and interpreting these instructions one at a time

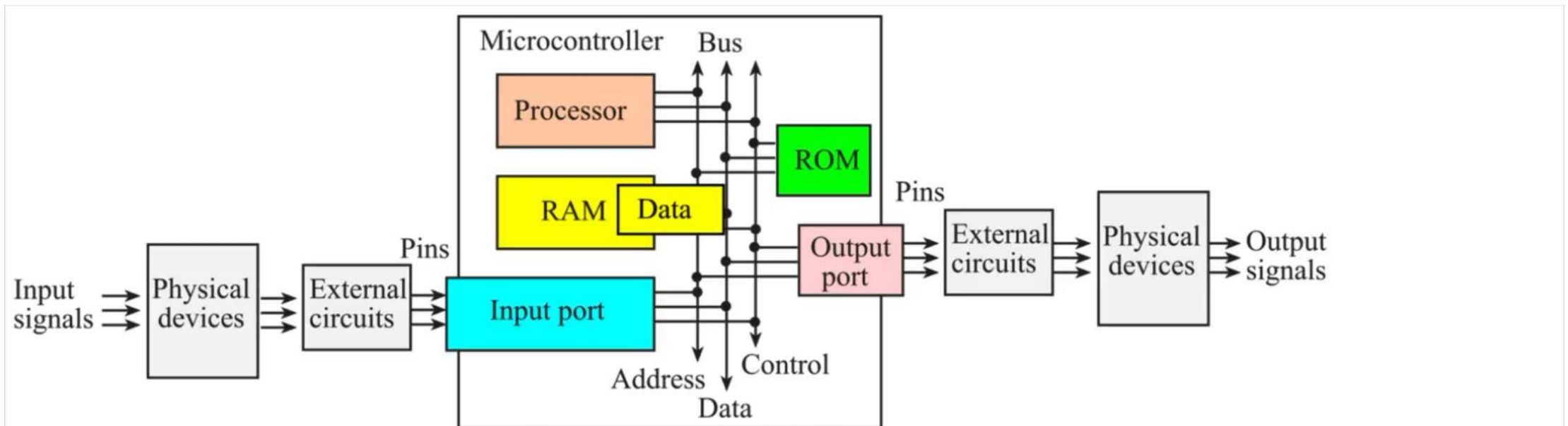


Figure 1.1.1. The basic components of a von Neumann computer including processor, memory and I/O connected with by a single bus.

Recap: Harvard architecture

- The processor can simultaneously fetch an instruction out of flash ROM using the ICode bus while it writes data into RAM using the system bus

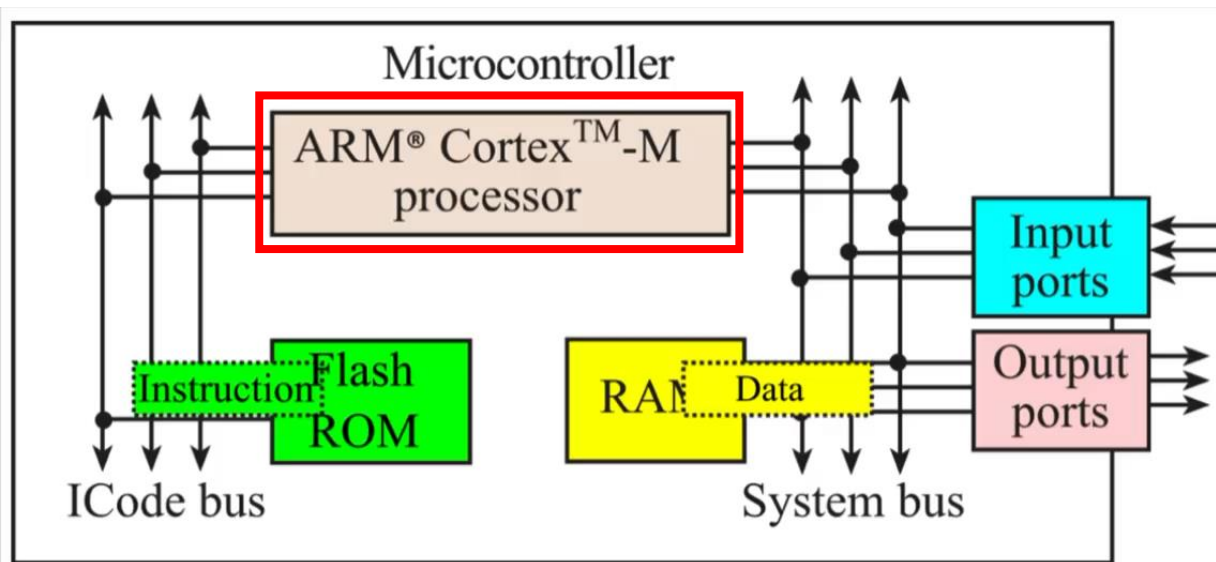


Figure 1.1.2. Harvard architecture of an ARM Cortex-M4. The three signals on each bus are address, data, control. If all these components exist on a single chip, it is called a microcontroller.

Organization

Organization [From Dictionary]

- ▶ Something made up of elements with varied functions that contribute to the whole and to collective functions; an organism
- ▶ A structure through which individuals cooperate systematically to conduct business

Embedded Software Organization

- ▶ Planning methodology to create a coherent and effective structure in a software project, by categorizing different software components according to their specific characteristics, allowing to construct software systems that are reusable and portable through different hardware platforms and applications

Why software organization?

Factors driving Embedded Systems Development:

- ▶ Demanding and intensive requirements
- ▶ Hardware complexity
- ▶ Reduced development times; products released to market as soon as possible
- ▶ Cost

Why software organization?

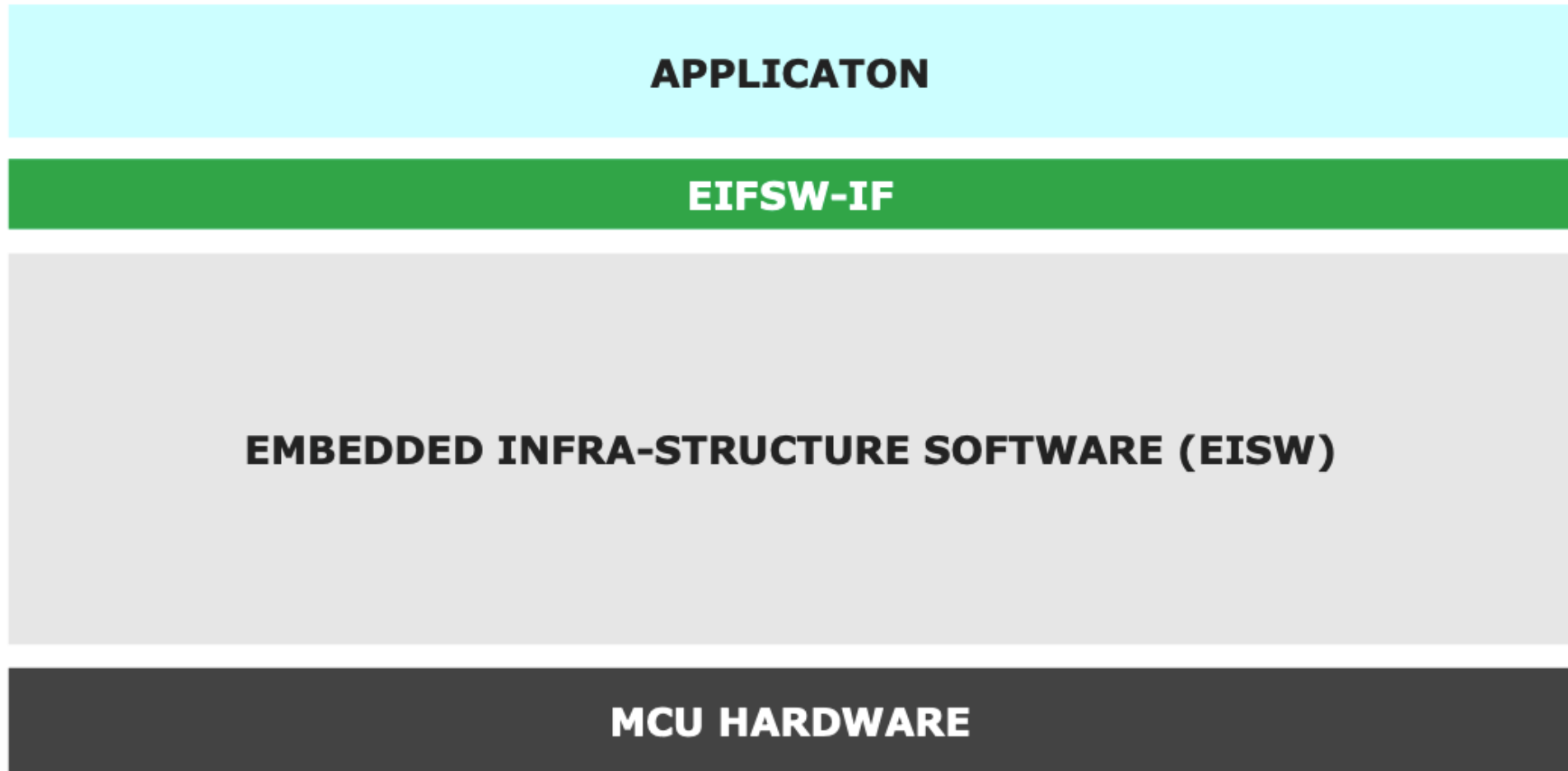
- ▶ The increasing complexity of system requirements as consequence of technology advancements in semiconductor industry
- ▶ Complex requirements critically impact the product life cycle. It is difficult to satisfy time-to-market demands (reduce development time and cost)
- ▶ Optimize and speed-up software development, without compromising safety, robustness and quality of the software components
- ▶ Improve software component reusability through multiple hardware platforms

Why software organization?

Embedded Systems Development Facts:

- ▶ Development time and engineering cost (i.e. NRE) become a problem, and the product falls behind schedule during the whole product life cycle
- ▶ Industry recognizes the importance of making an effort to optimize and speed-up software development, without compromise robustness and quality of the software components created to fit into a specific software architecture
- ▶ Software design and development are critical factors directly related to cost, time to market and the success of an embedded product
- ▶ The increasing complexity of embedded systems implies an increase in specialization of function in the design team [Ganssle, 2008]

A general view of embedded software architecture



Terms

Software Architecture

- ▶ The software architecture of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them. [Bass, Clements & Kazman, 2003]
- ▶ Collection of software components that follows an organized structure, and describes the overall system and its components' behavior from a high-level design perspective

Embedded Software Architecture

- ▶ Structure and organization of multiple software components through different abstraction layers that intend to provide hardware independence, maximize code reusability and propagate component behaviors, between multiple platforms of purpose-specific embedded computers

Terms

Abstraction

- ▶ Simplified view of a system containing only the details important for a particular purpose [Berzins & Luqi, 1991]

Embedded Software Abstraction

- ▶ Design methodology used to hide hardware architecture details from the application software domain by the isolation and encapsulation of relevant parameters that describe the behavior of an specific hardware entity, in order to facilitate software component reusability and portability

Software Component

- ▶ In software system, a software component is an entity with well defined behavior and interacts with other components and modules within the system

Terms

Software Interface

- ▶ A mechanism used by a software component or module to interact with the external world (i.e., analog/digital signals, RF) and other software components

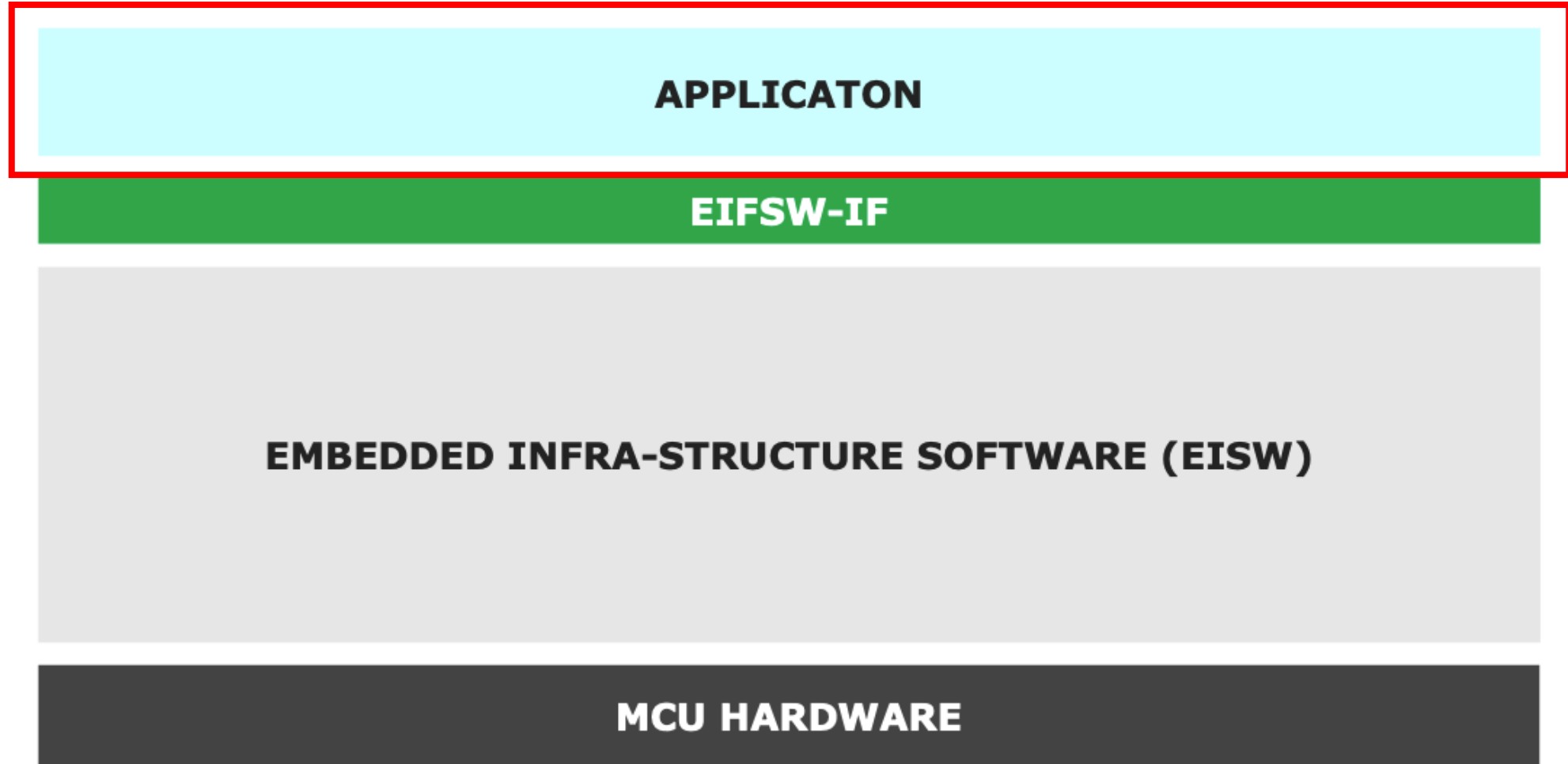
Coupling

- ▶ Degree of dependency between different software components within a system

Cohesion

- ▶ Measures the degree of relationship between elements within a software component.

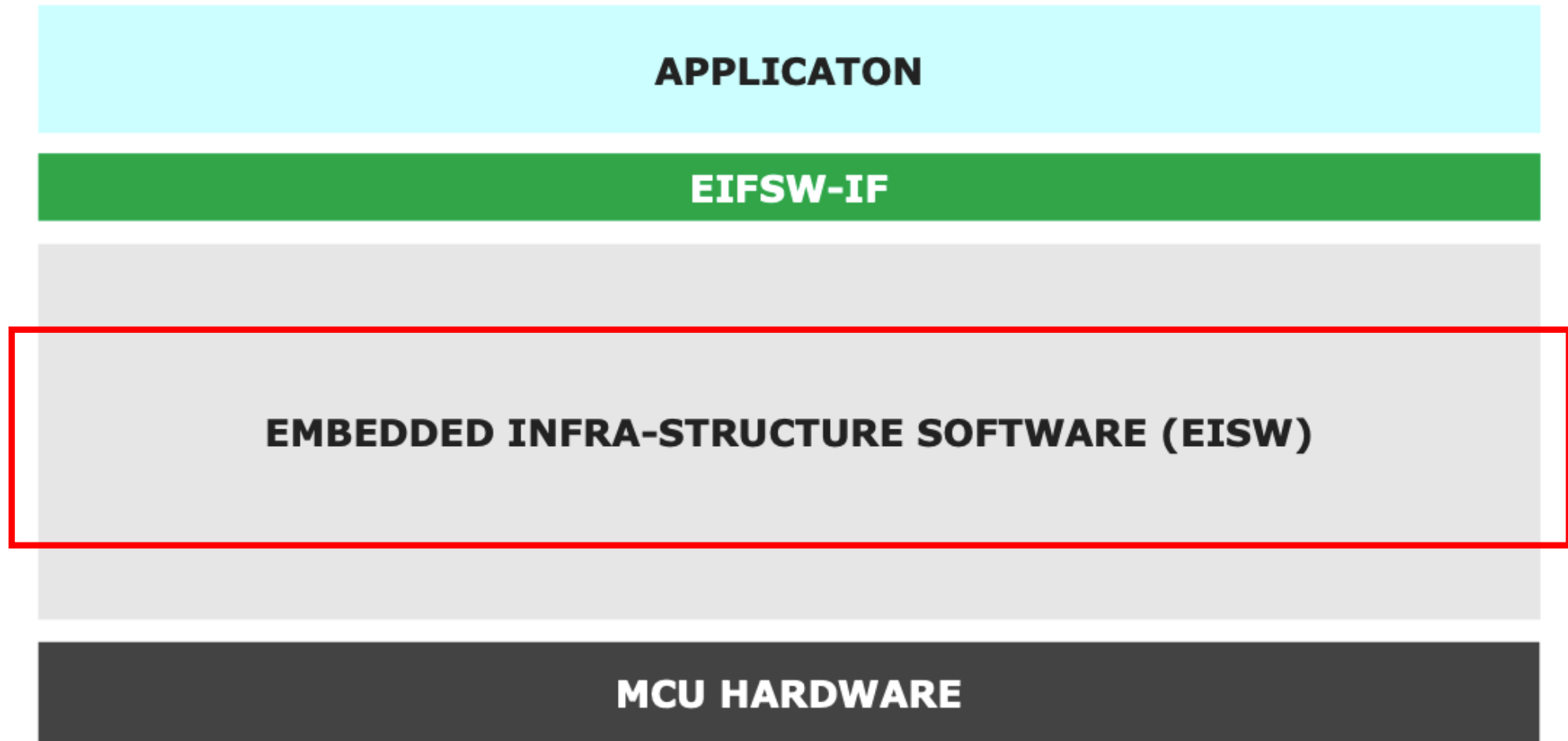
Application



Application

- The high-level functional software that implements vehicle- or system-specific features (e.g., motor control, thermal management).
- Product specific functions
- Contains the software components that implements the desired functionality (unique) for a specific embedded computer system
- Hardware-agnostic and relies on services provided by lower layers.
- Reusability of application components strongly depends in the availability and efficiency of lower layers

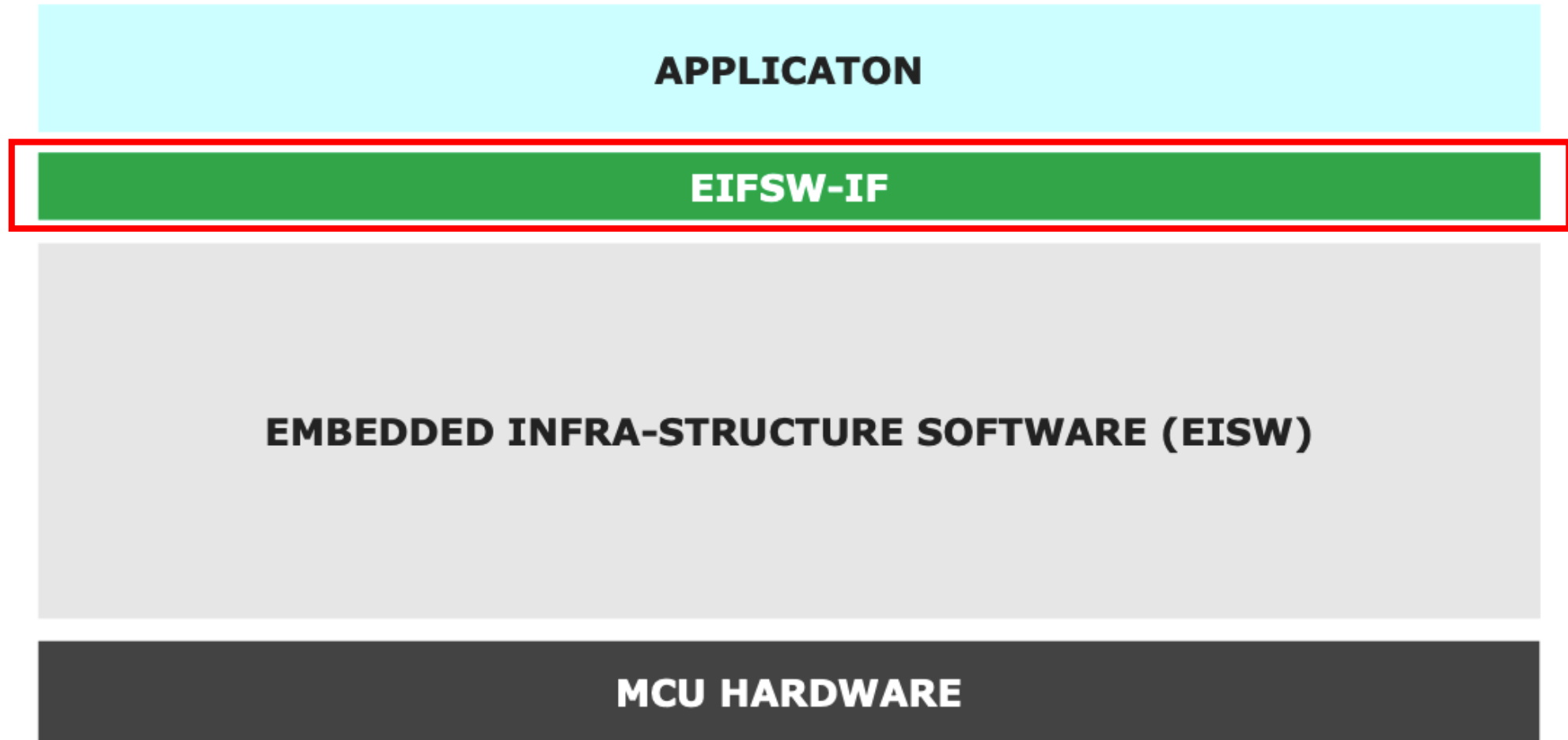
EISW (Embedded infrastructure software)



EISW (Embedded infrastructure software)

- The runtime foundation that provides essential services to the Application.
- Includes: Operating system (e.g., OSEK/RTOS) Communication stacks (CAN, LIN, Ethernet) Diagnostics (UDS, DCM) Memory services I/O abstraction Safety & security middleware
- May also integrate MCAL (Microcontroller Abstraction Layer) to interface with hardware.
- Key role: Provides reusable, standardized, and often certified services across projects.

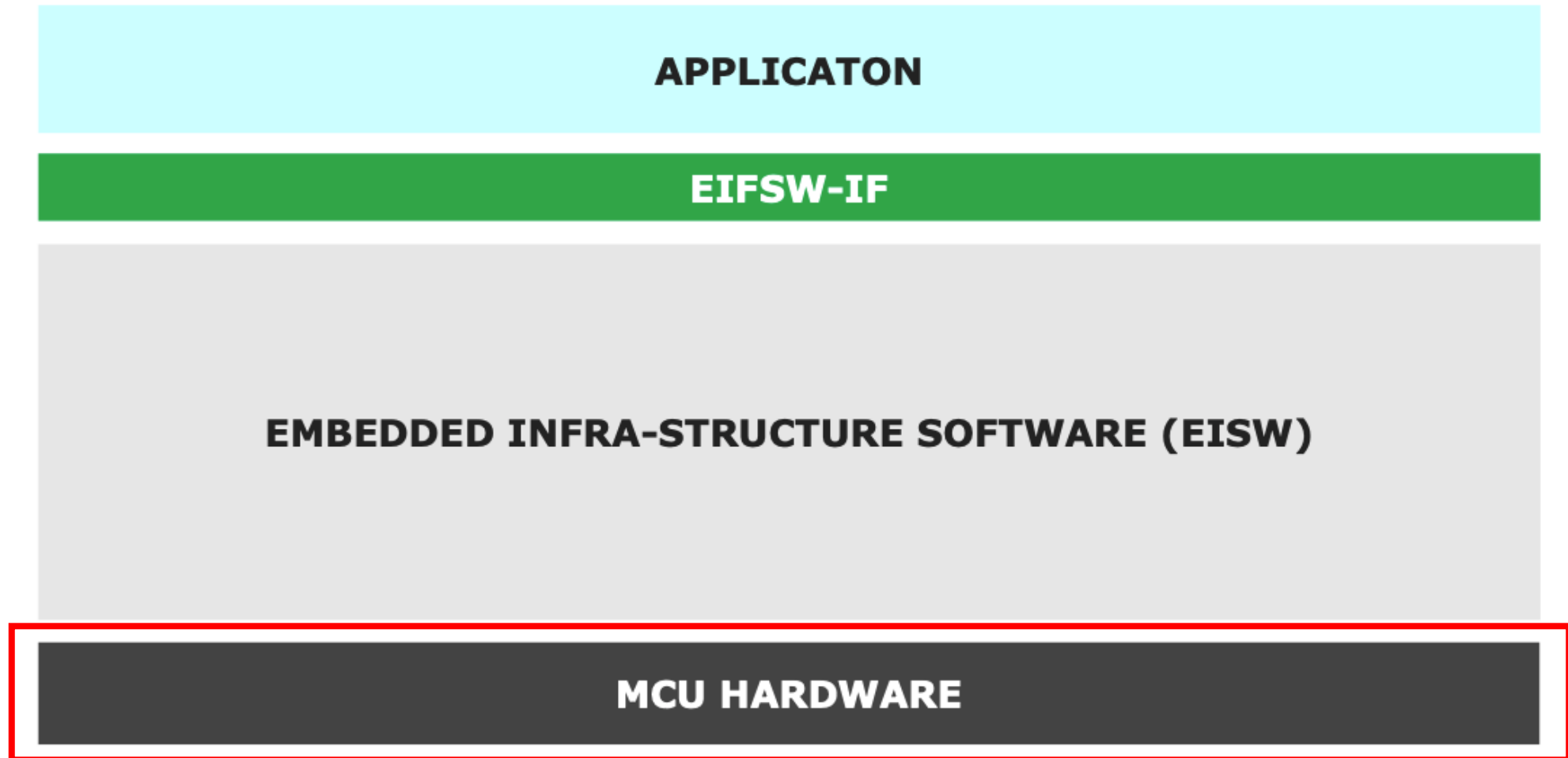
EIFSW-IF (Embedded infrastructure software – Interface)



EIFSW-IF (Embedded infrastructure software – Interface)

- Provides to the application one more level of abstraction and hardware independence
- Translates logical signals into a meaningful format for the application
- Facilitates the communication between application software components and/or lower layer modules
- It is application specific

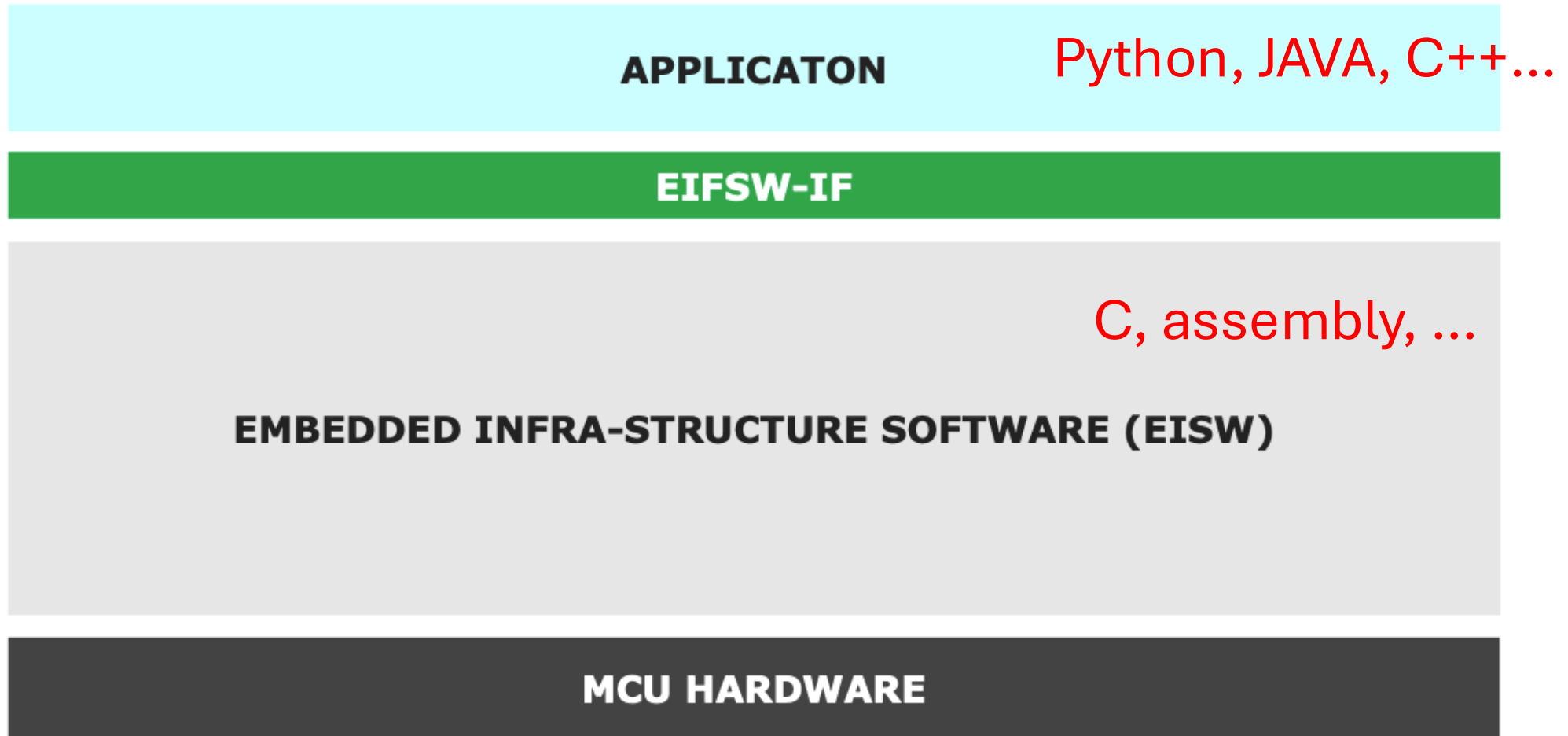
MCU hardware



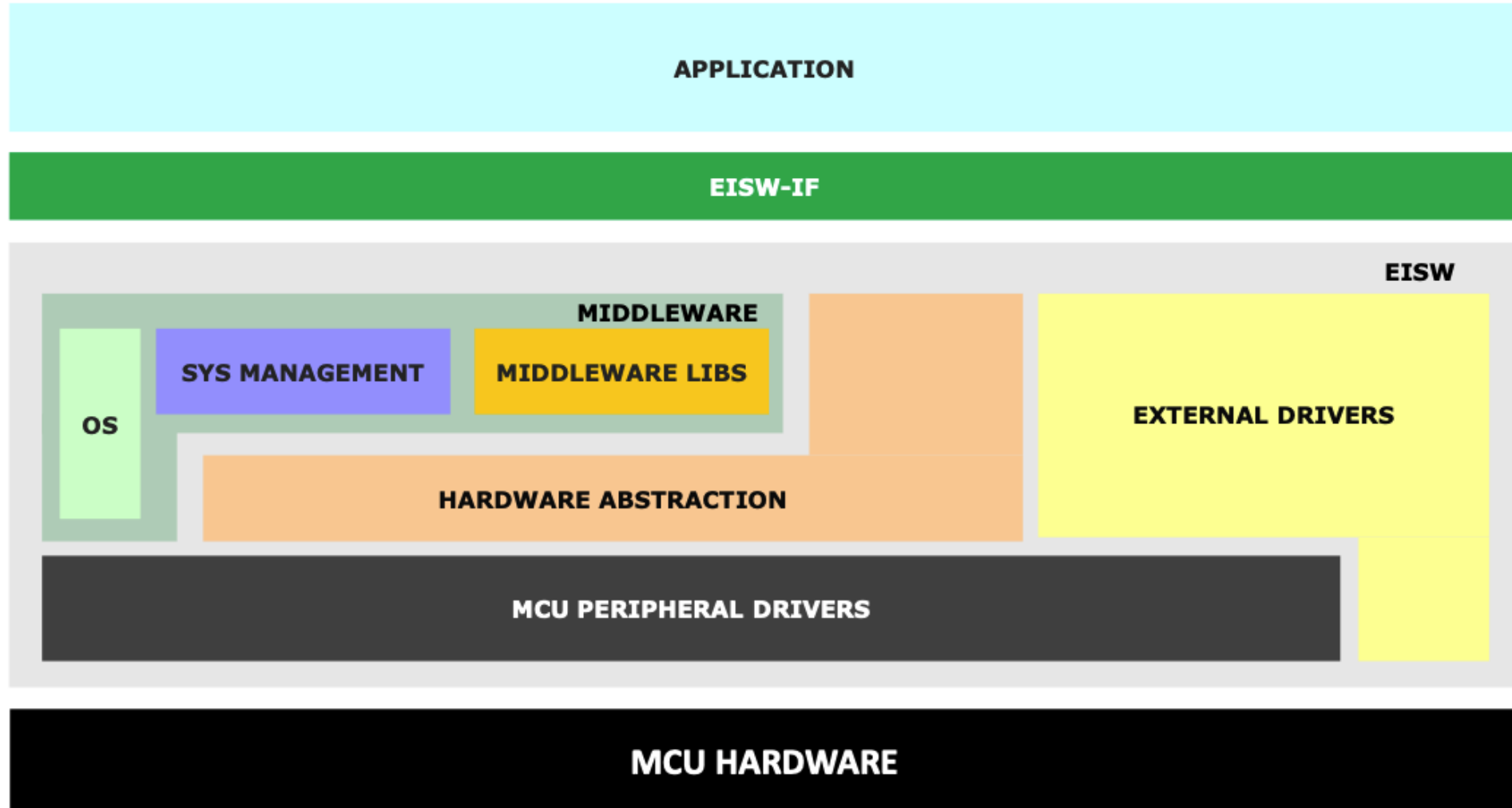
MCU hardware

- The physical microcontroller (e.g., Infineon TC3xx, NXP S32G, Renesas RH850).
- Executes all software layers.
- Directly accessed only by the lowest layers of EISW.

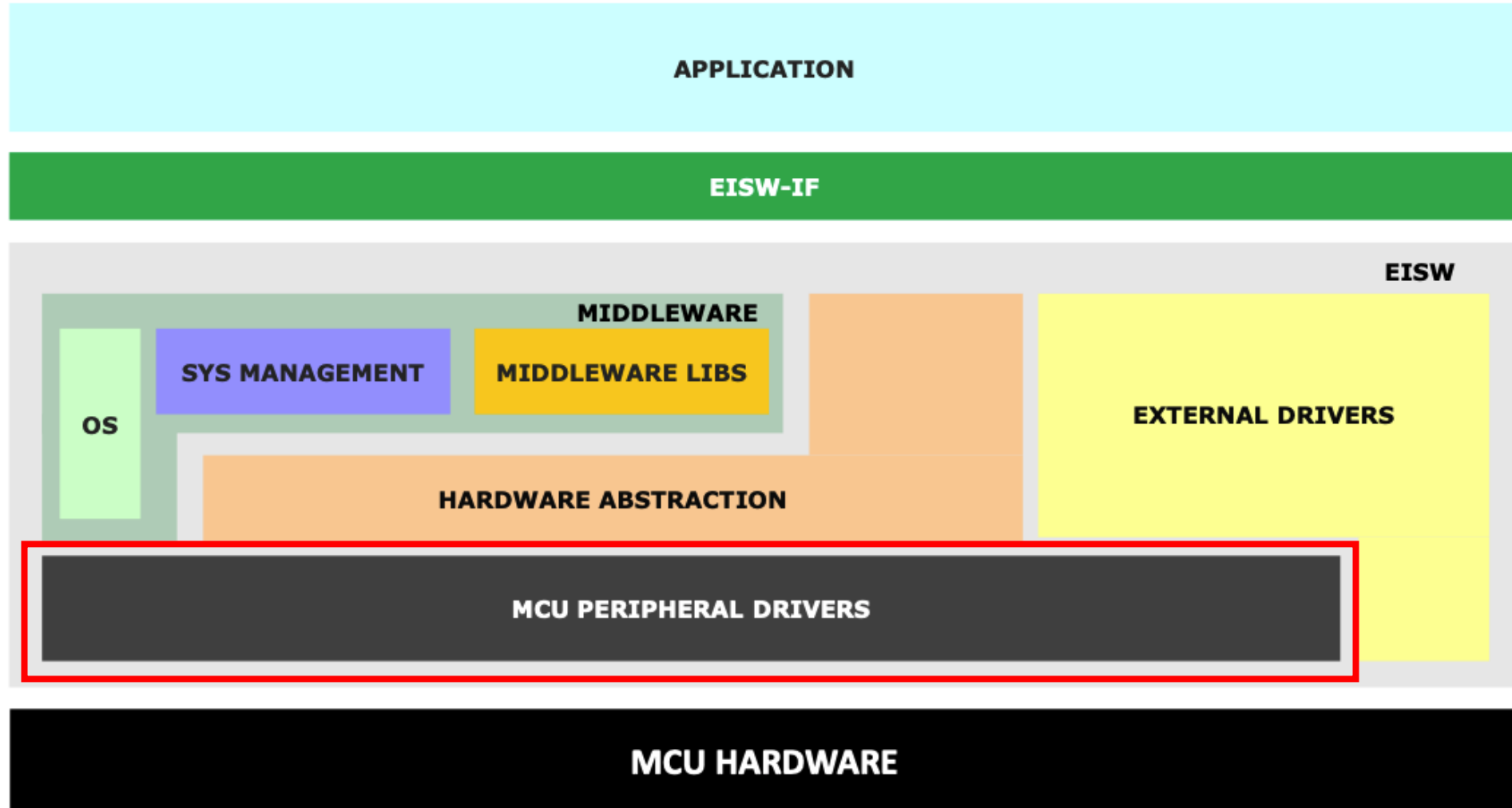
A general view of embedded software architecture



More within EISW



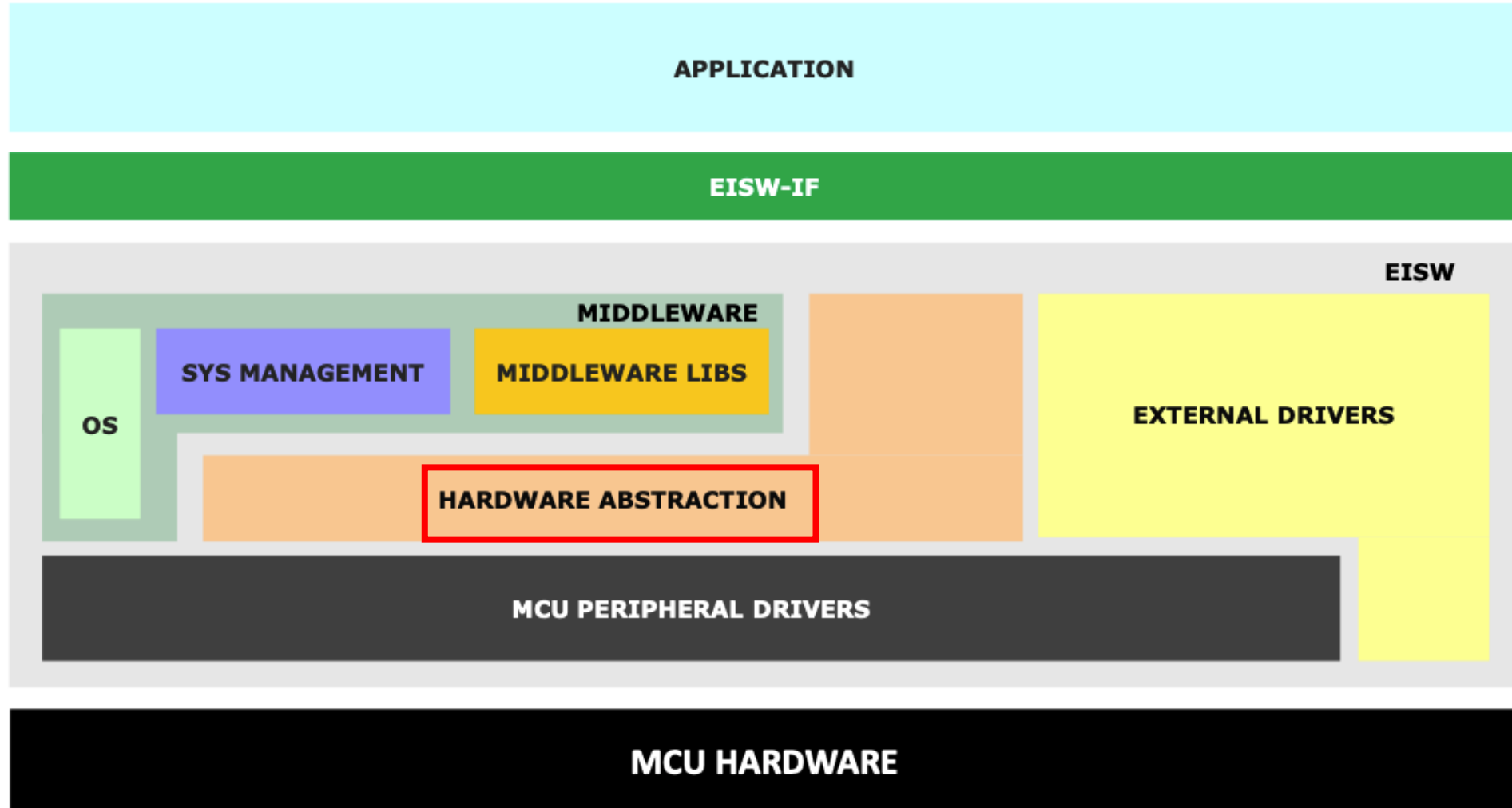
MCU peripheral drivers



MCU peripheral drivers

- Internal device drivers
- Hardware access to MCU peripherals
- Provide MCU low-level abstraction
- Hardware dependence is high, therefore, reuse is limited at this level
- Provide standard interfaces used by abstraction, OS and external driver layers

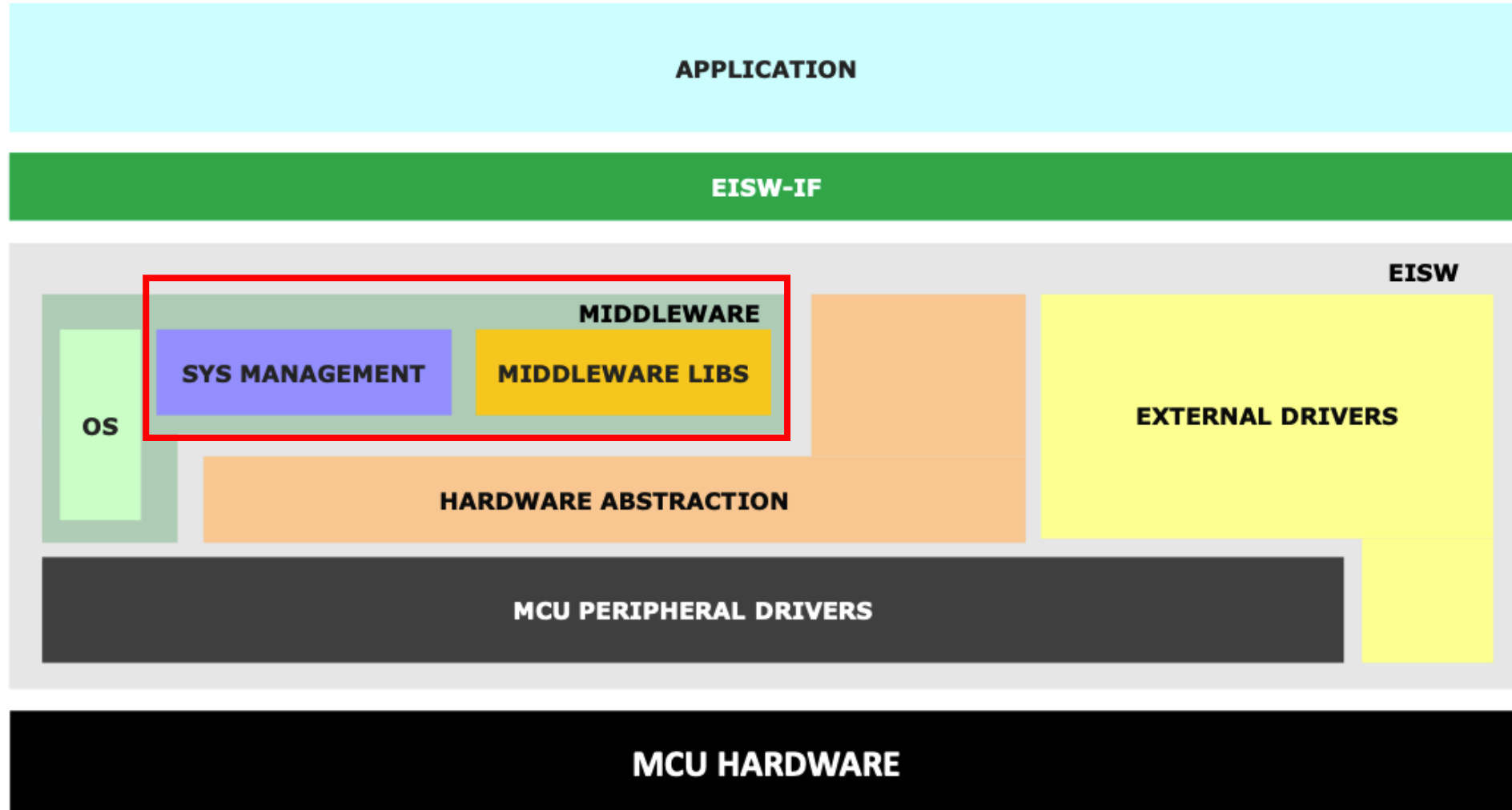
Hardware abstraction layer (HAL)



Hardware abstraction layer (HAL)

- Provides access to MCU hardware features through peripheral interfaces
- Hides hardware details not relevant to upper software layers
- Interfaces with MCU and external drivers in the low level side, and with HAL signal interface at the upper side

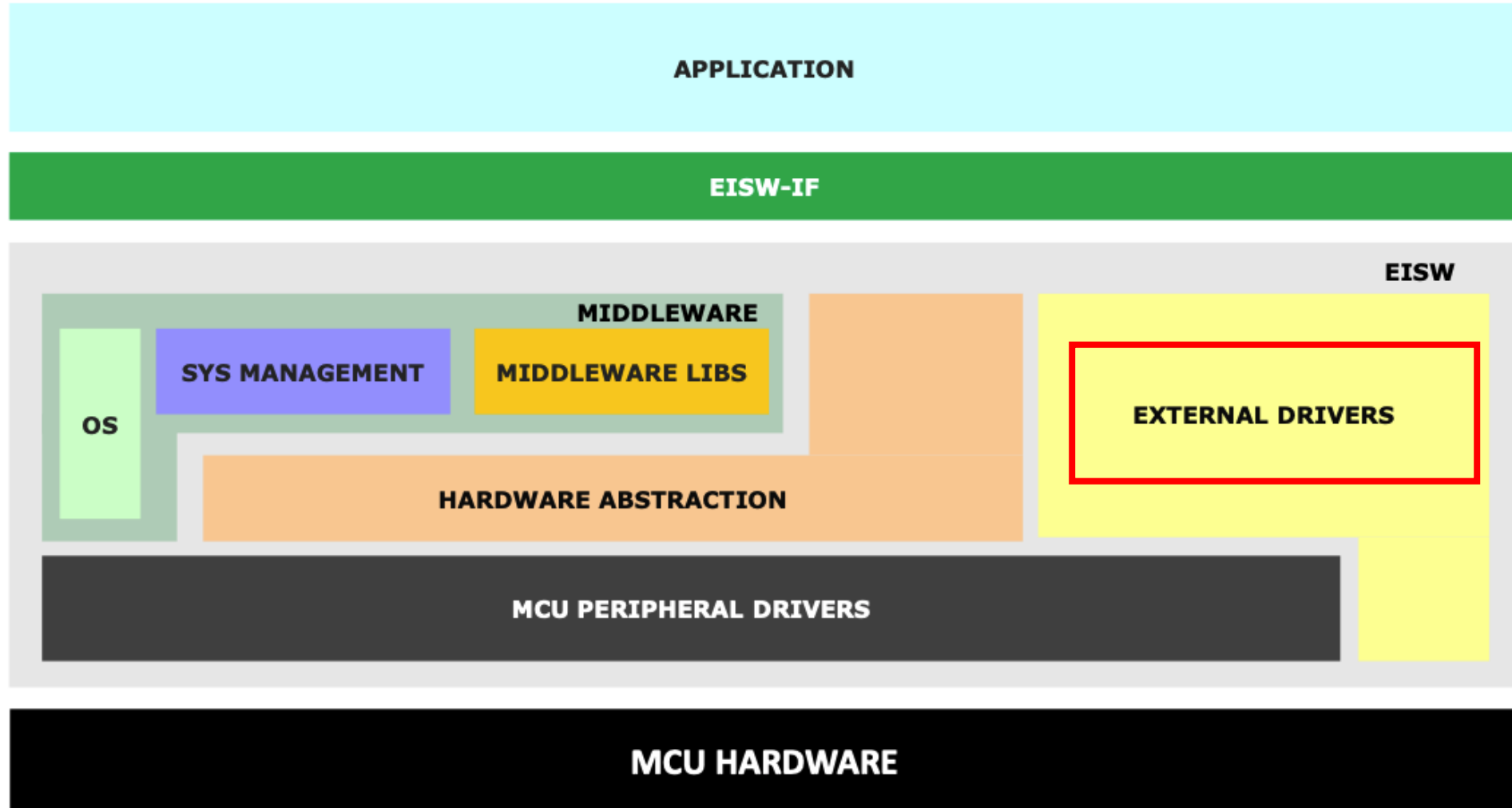
Middleware and system management



Middleware and system management

- ▶ Facilitate the interaction between application components and other modules and/or components within the system:
 - > Graphics Library
 - > Networking
 - > File Systems
 - > Databases
 - > Other Middleware components, i.e., off-the-shelf components
- ▶ Provides system management
 - ▶ Power Management
 - ▶ Memory management
 - ▶ Diagnostics
- ▶ Due to overhead, it is an optional layer

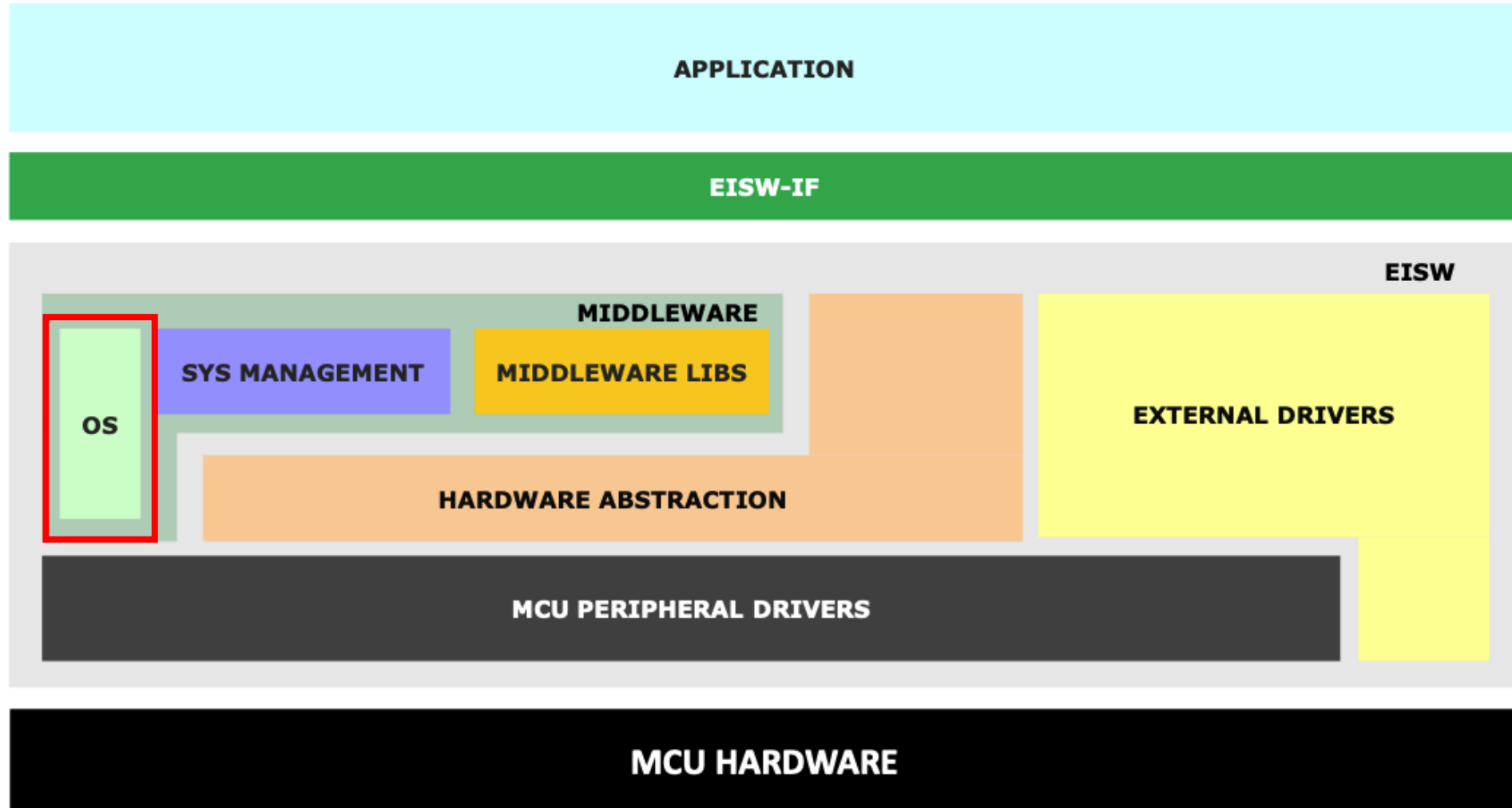
External drivers



External drivers

- ▶ Implements direct hardware access to external devices through MCU peripherals
- ▶ Meet all functional and timing requirements of the external devices
- ▶ Examples:
 - ▶ EEPROM (I2C™, SPI™, Microwire™, etc)
 - ▶ External ADCs (i.e. Delta-Sigma high-resolution converters)
 - ▶ Sensors and actuators
 - ▶ System Basis Chip (SBC)
 - ▶ I/O Driver ICs

OS



OS

- Provides support for multi-tasking
- Task scheduling and synchronization
- If real-time OS (RTOS)
 - Context –switching
 - Task preemption
 - Interrupt management

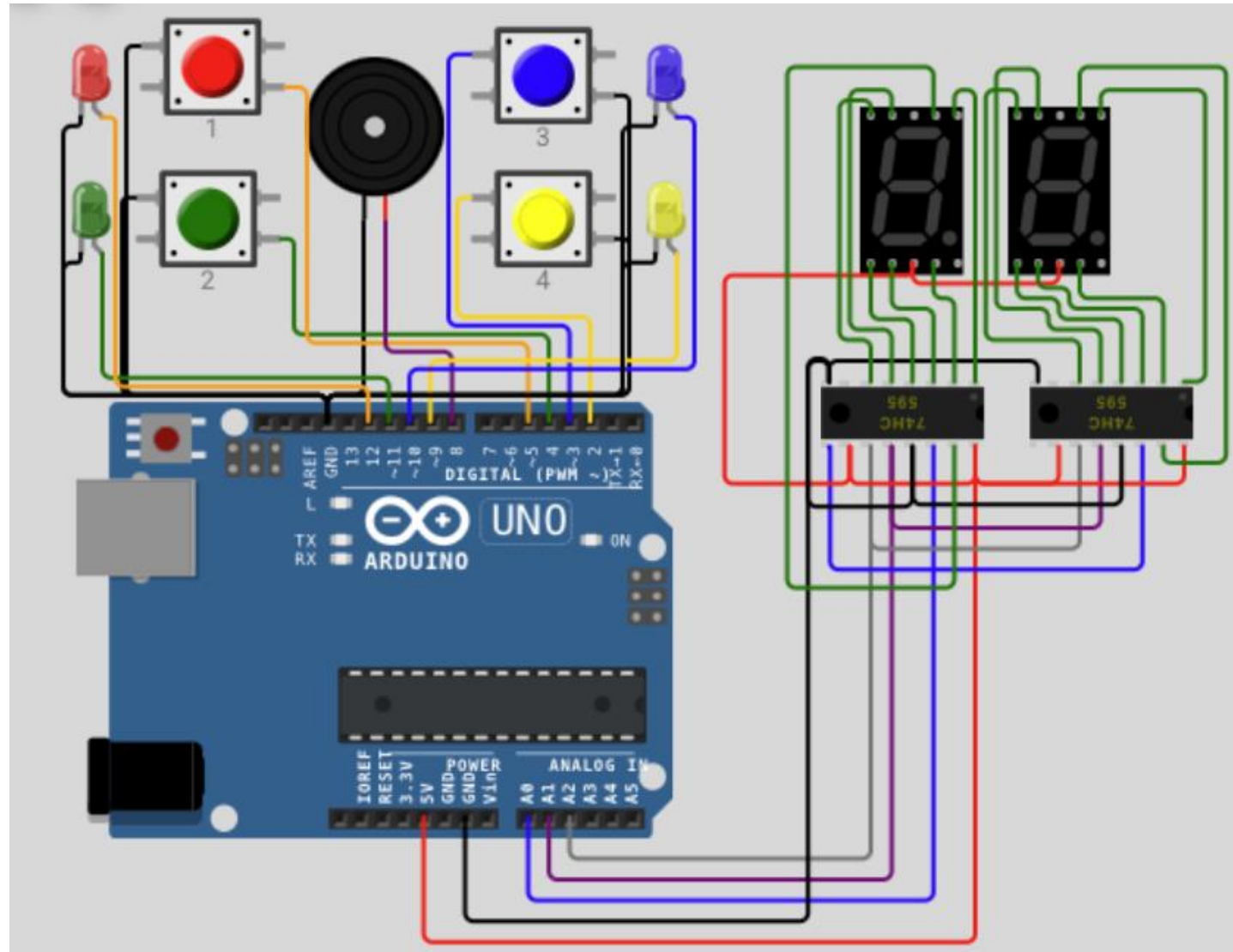
Advantages of software layered architecture

- ▶ Organized and modular design
- ▶ If properly applied, an architectural approach allows and facilitate distributed development; software components being developed by different teams or COTs from third parties (modular and scalable)
- ▶ Once a software architecture has **evolved** reaching an optimal level of **maturity**, the development process can be benefited by reducing development time and cost.
- ▶ Well defined architectures facilitate the usage of more advanced development techniques and tools, i.e., Model Based and Code Generation

Disadvantages of software layered architecture

- ▶ Modular layered software architectures and abstraction can consume significant resources in an embedded system in terms of memory and performance:
 - > From few kilobytes of ROM/RAM to the order of several megabytes
 - > From tenths of MHz to hundreds of MHz (even GHz)
- ▶ Transitioning from traditional embedded software development into a layered software architecture, can result in a large learning curve:
 - > Adopting a new design and implementation methodology
 - > Learning new tools
- ▶ Initially, the adoption of software layered architectures may result in a spike in cost and development time, making difficult its acceptance

Virtual lab session recap: Simon game



Virtual lab session: Simon game

- Can you write down the software organization of simon game?

Virtual lab session: Simon game

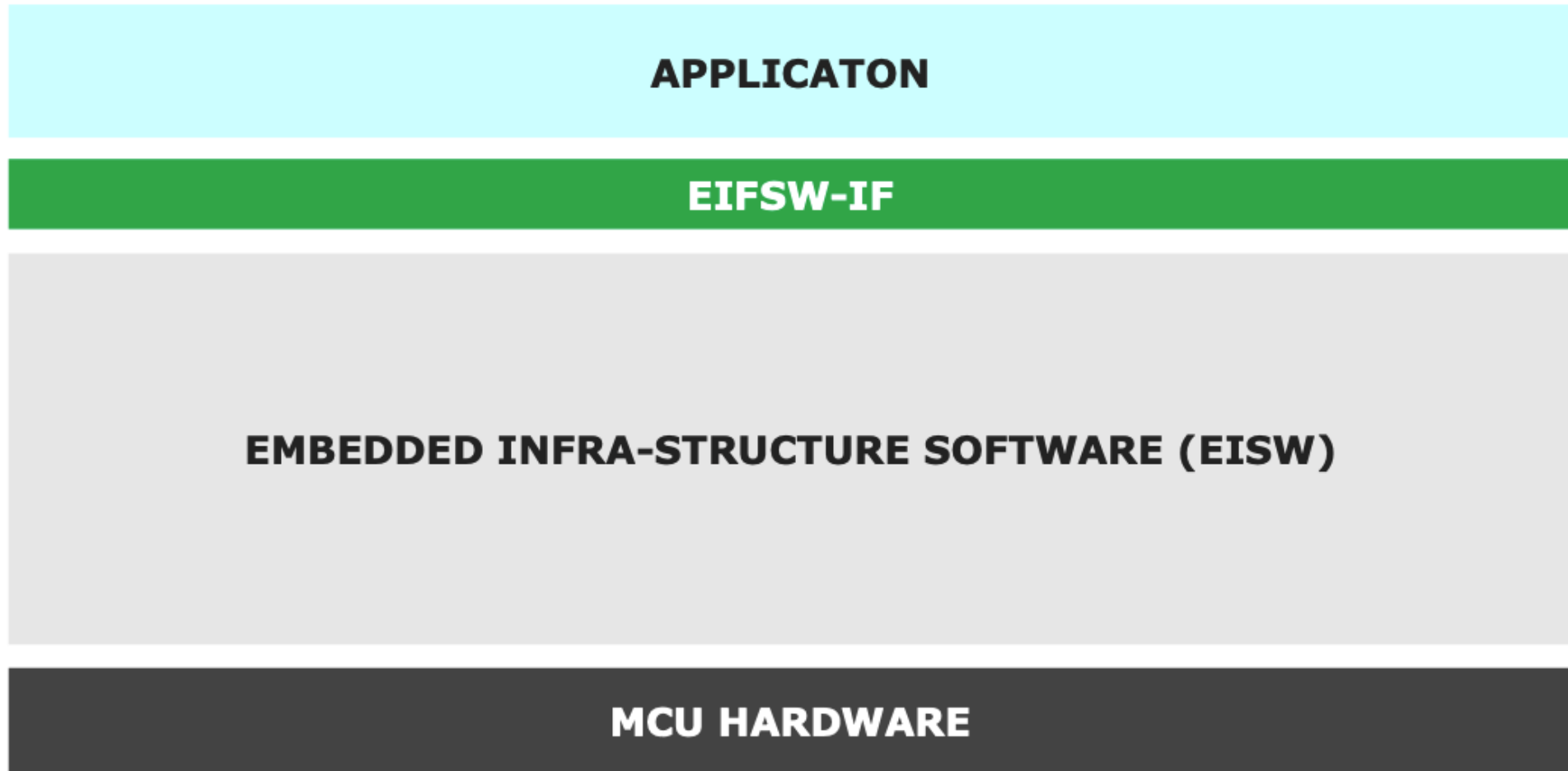
- Can you change the 4-LED Simon game to 6-LED Simon game?

Embedded software organization, design and development process

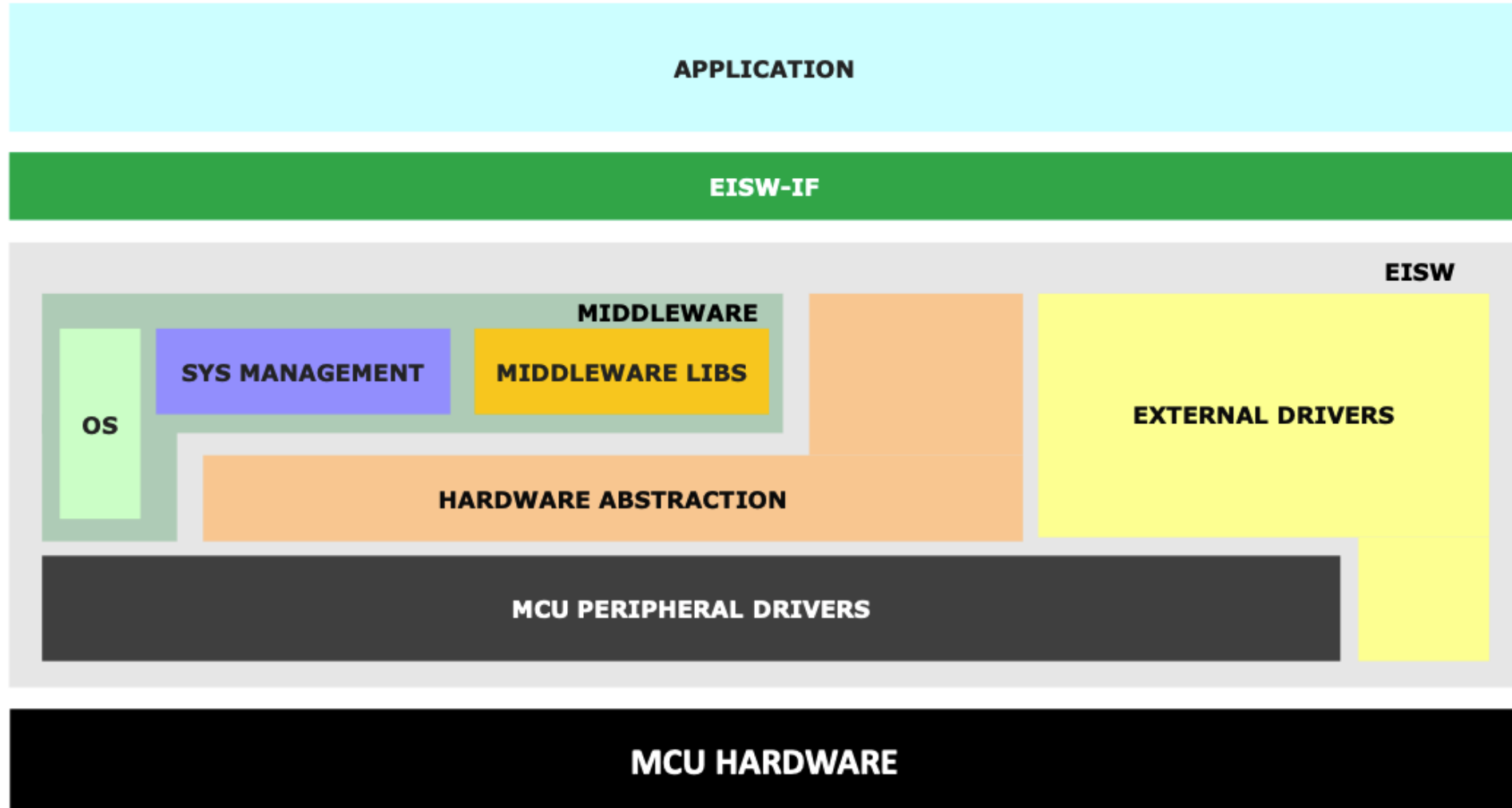
Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Recap: A general view of embedded software architecture



Recap: More within EISW



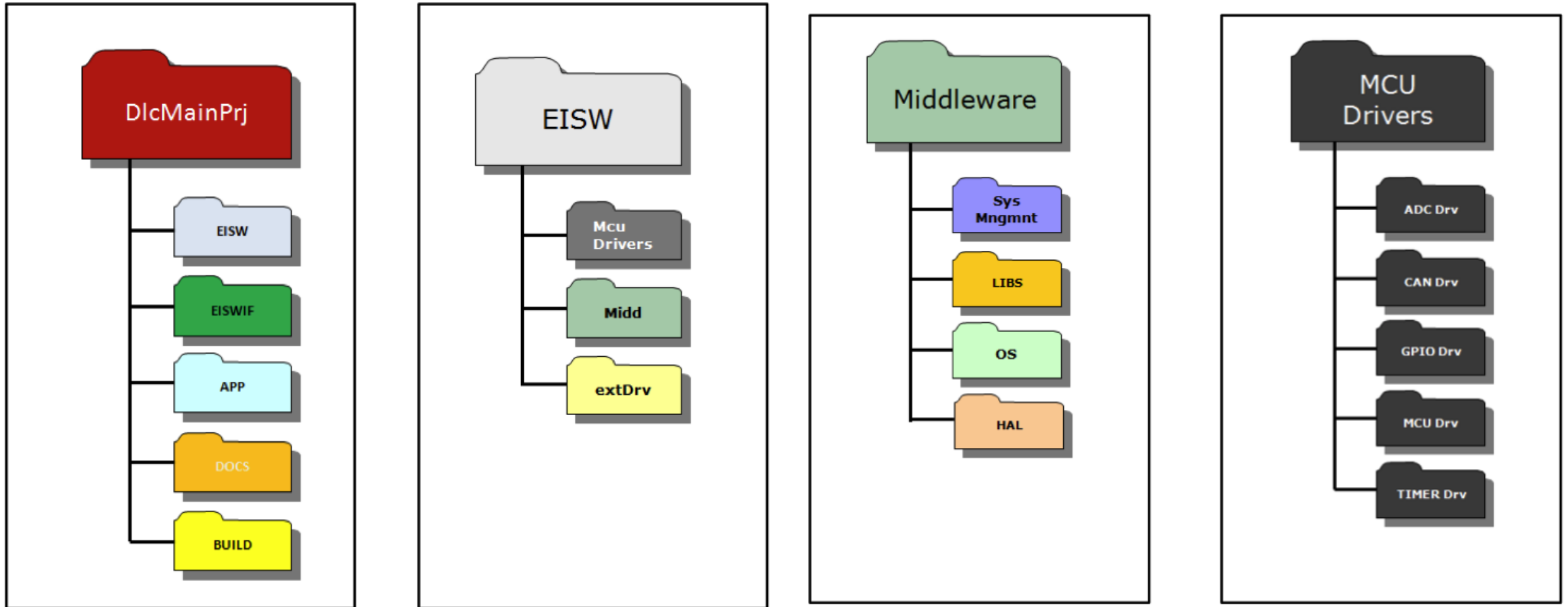
Recap: Advantages of software layered architecture

- ▶ Organized and modular design
- ▶ If properly applied, an architectural approach allows and facilitate distributed development; software components being developed by different teams or COTs from third parties (modular and scalable)
- ▶ Once a software architecture has **evolved** reaching an optimal level of **maturity**, the development process can be benefited by reducing development time and cost.
- ▶ Well defined architectures facilitate the usage of more advanced development techniques and tools, i.e., Model Based and Code Generation

Recap: Disadvantages of software layered architecture

- ▶ Modular layered software architectures and abstraction can consume significant resources in an embedded system in terms of memory and performance:
 - > From few kilobytes of ROM/RAM to the order of several megabytes
 - > From tenths of MHz to hundreds of MHz (even GHz)
- ▶ Transitioning from traditional embedded software development into a layered software architecture, can result in a large learning curve:
 - > Adopting a new design and implementation methodology
 - > Learning new tools
- ▶ Initially, the adoption of software layered architectures may result in a spike in cost and development time, making difficult its acceptance

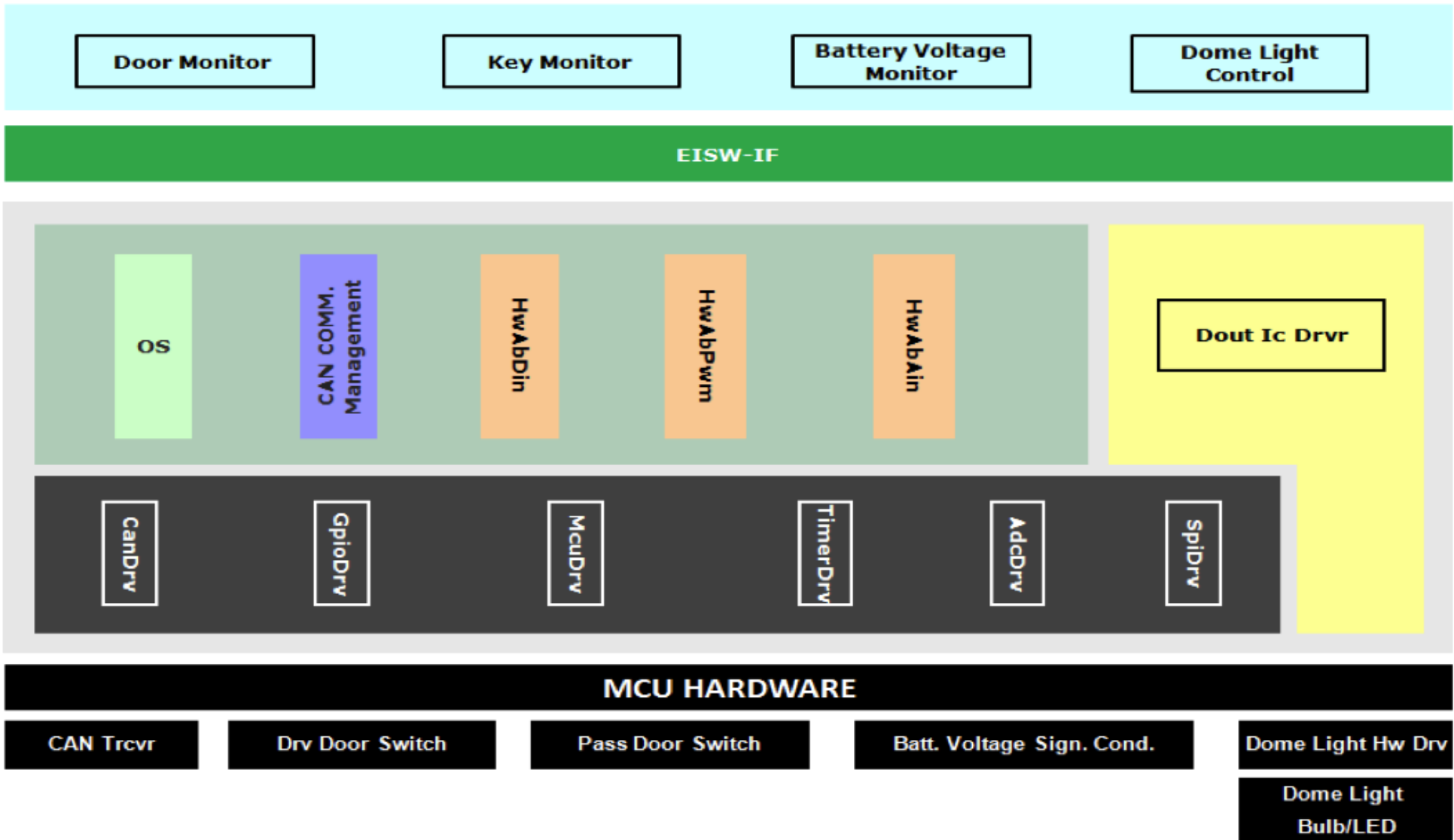
Directory structure



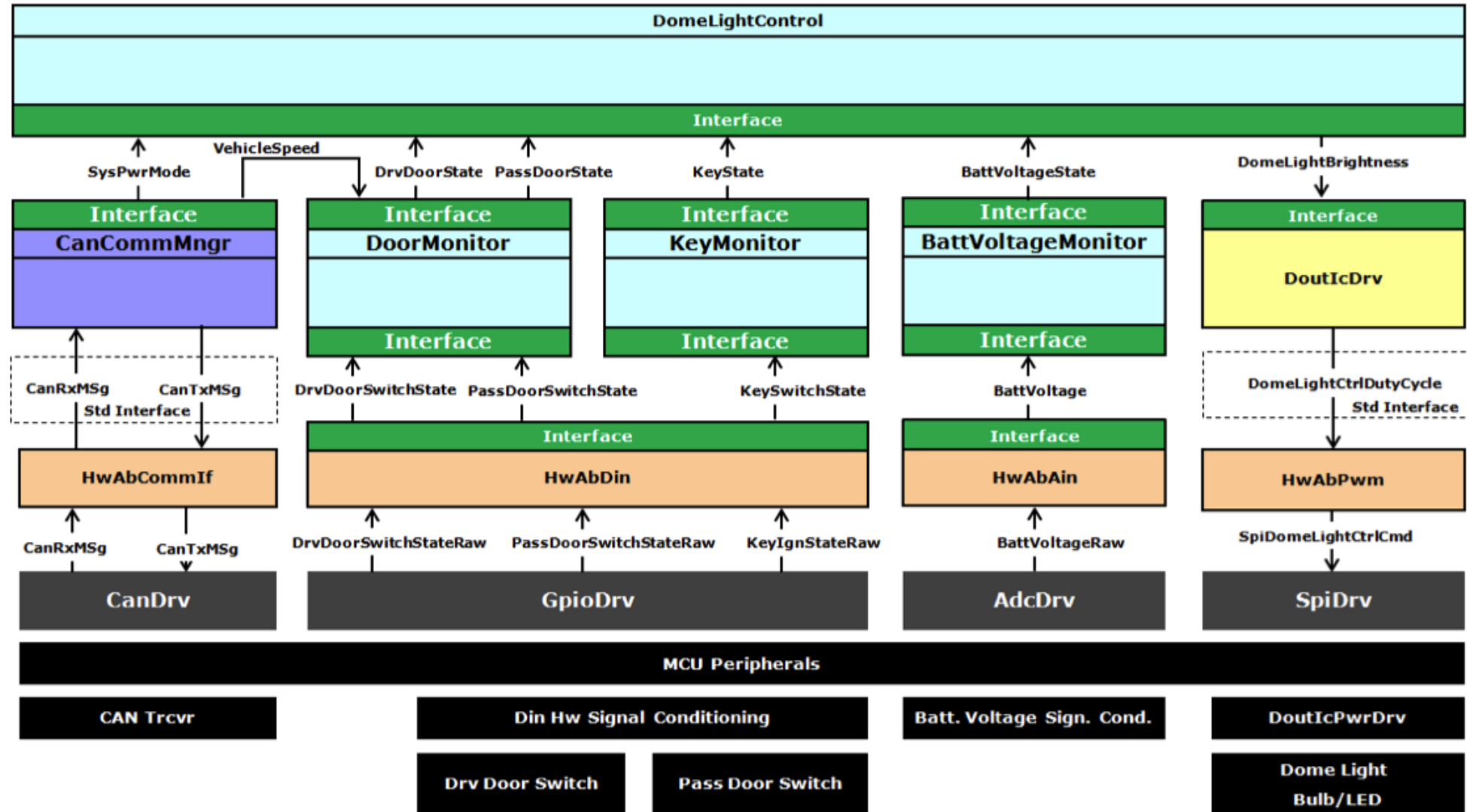
Example: Dome light control



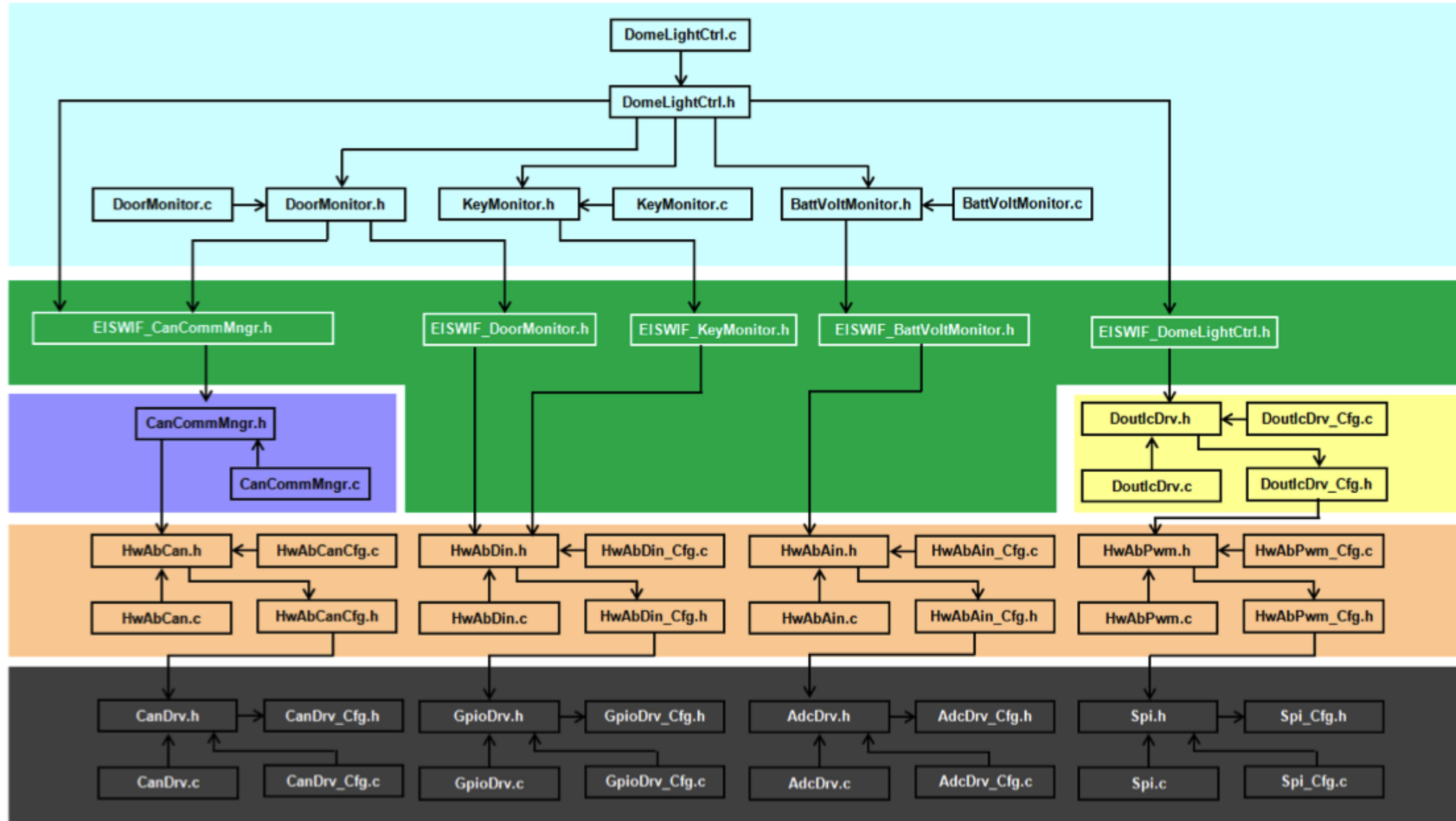
Software layers organization



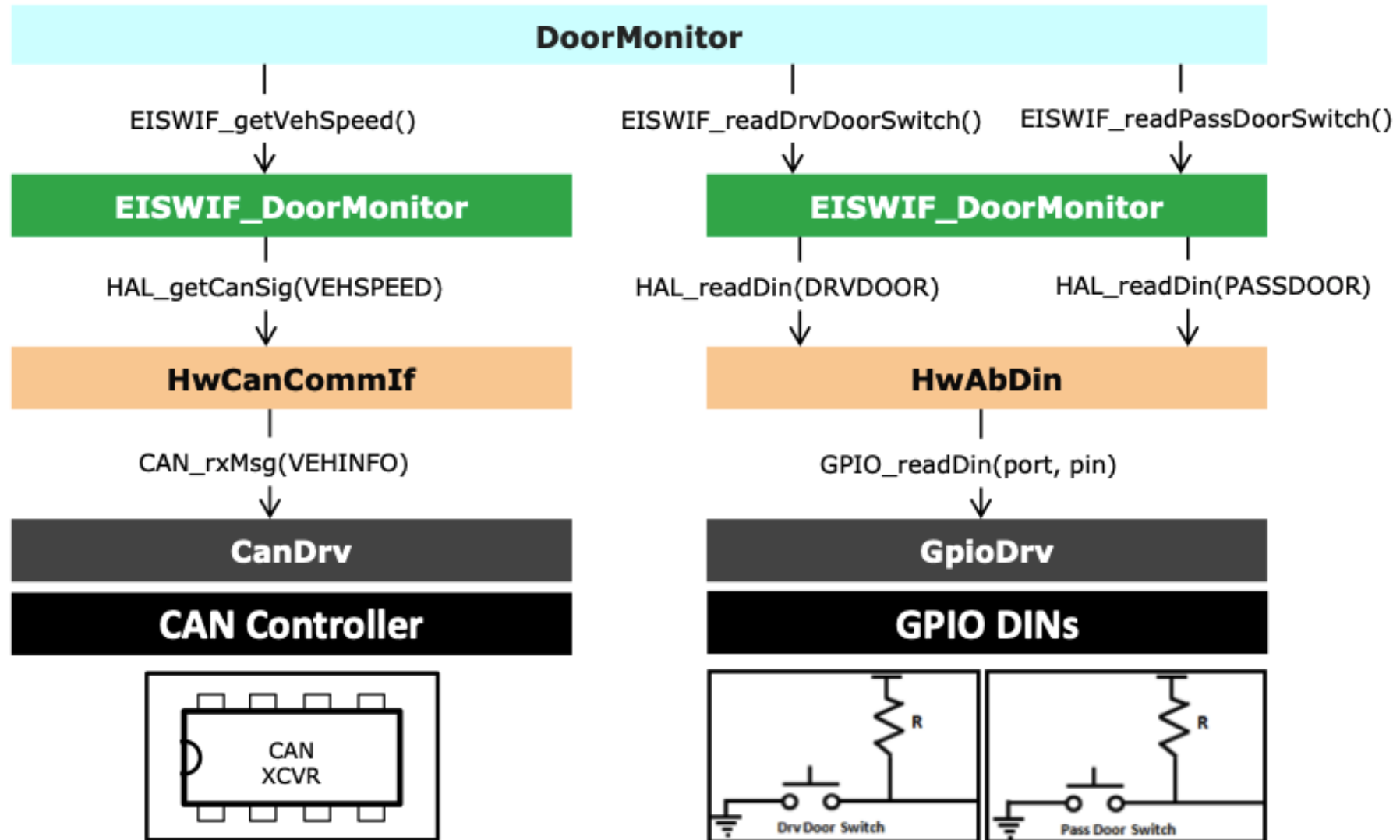
Component interaction



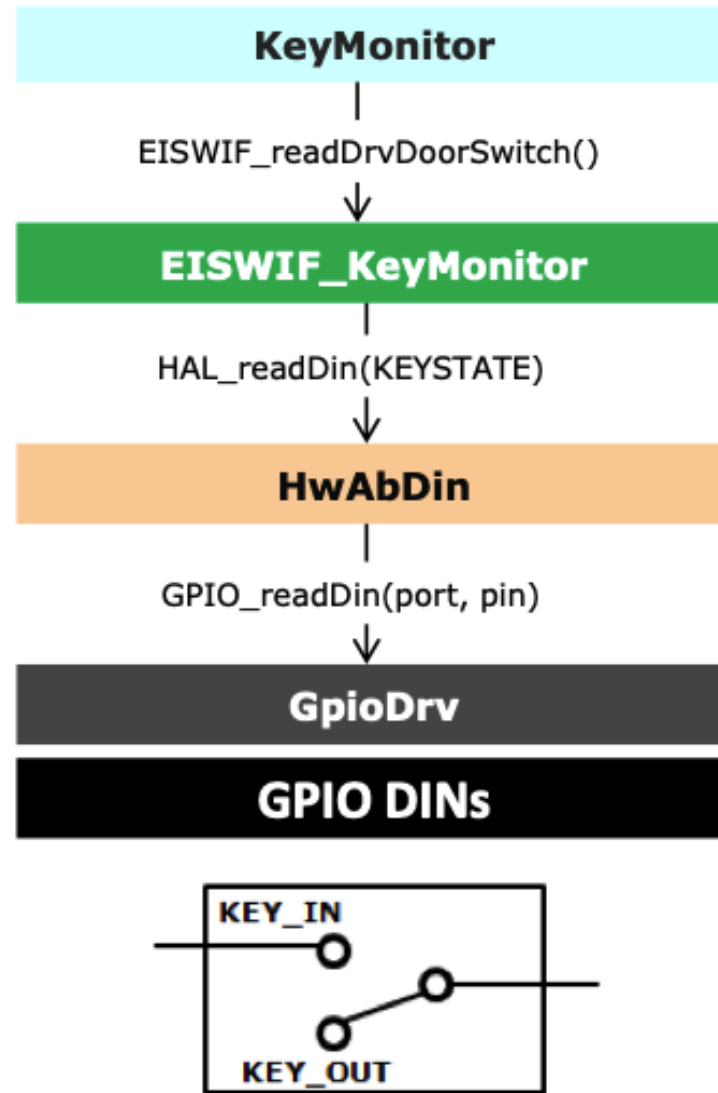
File structure



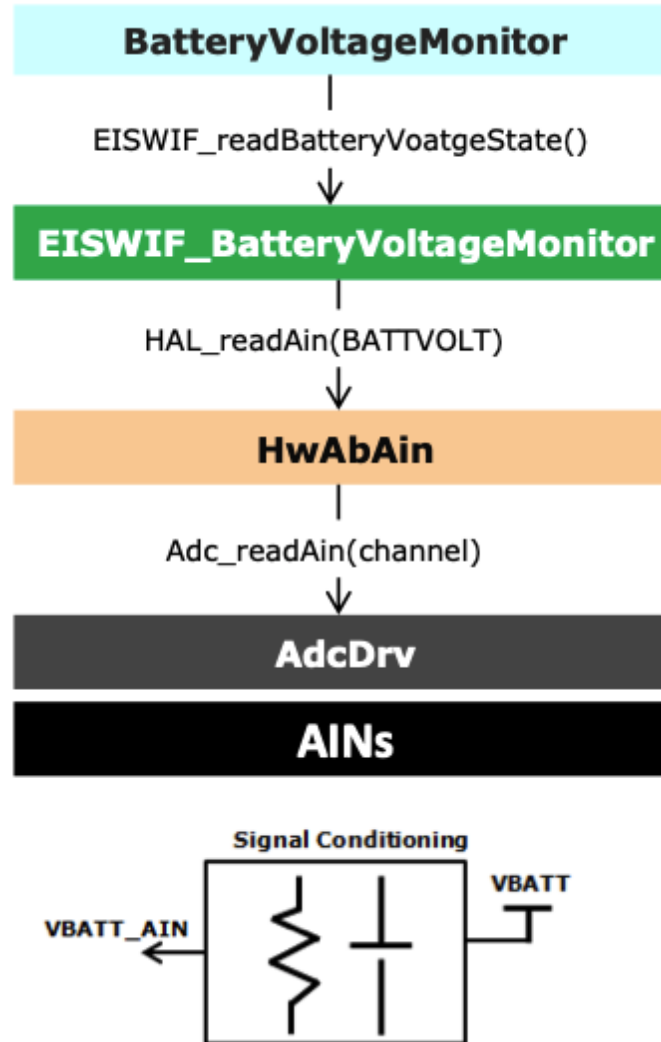
Door monitor component interaction



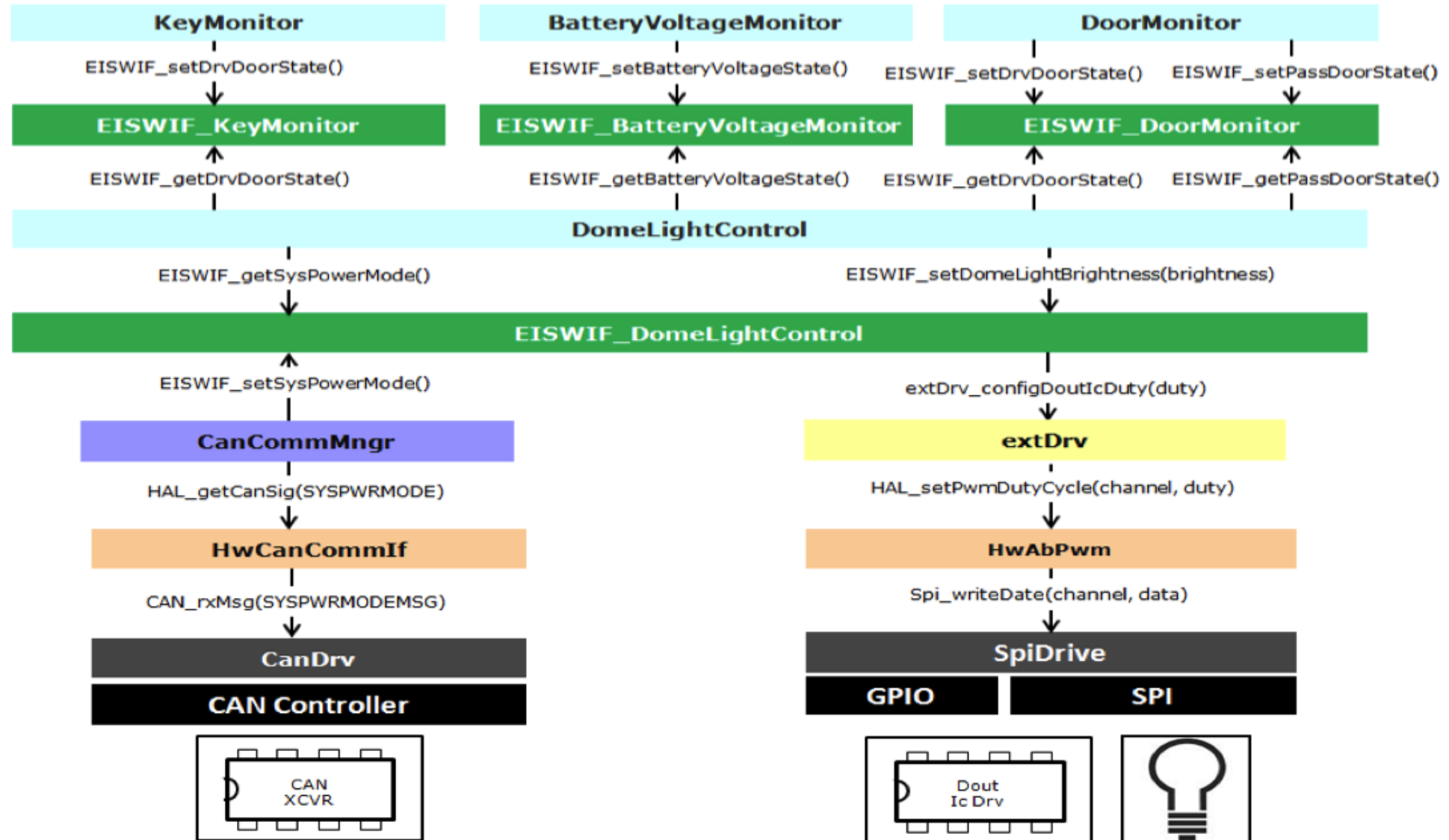
Key monitor component interaction



Battery voltage monitor component interaction



Dome light control monitor component interaction



So... all this to turn-on a light?

- ▶ The embedded world requires more sophisticated design methodologies that can satisfy current market demands
- ▶ The even more exigent and complex requirements trigger the advances of semiconductor industry, providing more powerful processors; therefore, software becomes more complex as well
- ▶ The evolvement of software requires the application of more advanced software engineering concepts
- ▶ Embedded Software development is not the task of electrical engineers anymore; neither pure software engineers; new culture of development is emerging

Five Phases Define the Path from Problem to Deployment

1. Analysis

Discover requirements and constraints

2. Design

Build conceptual models and specifications

3. Development

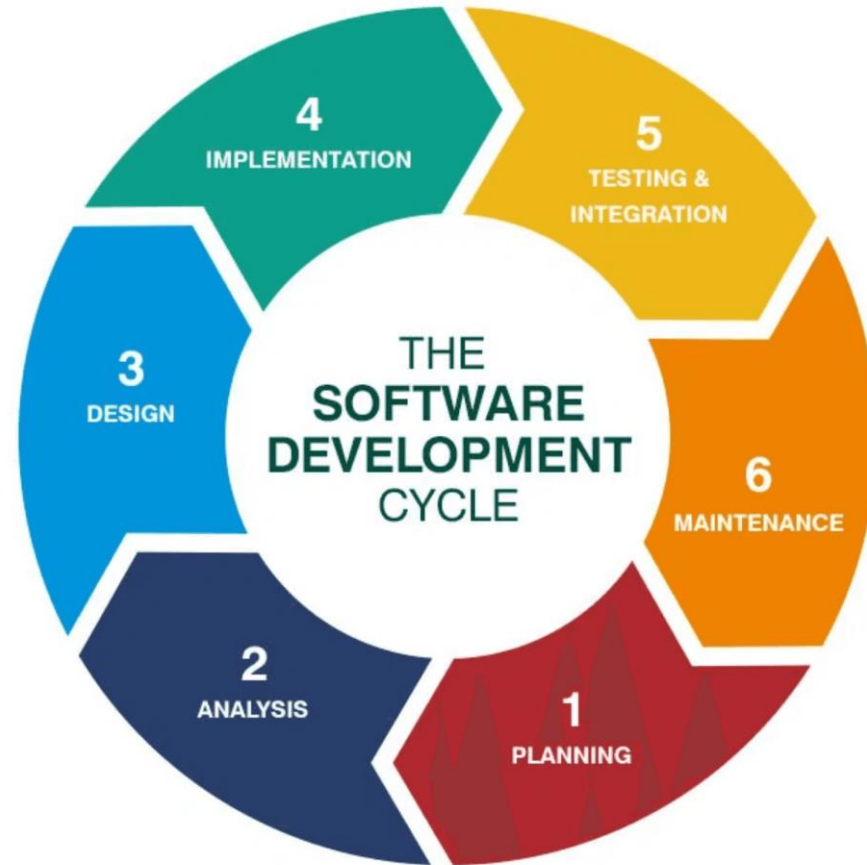
Implement hardware and software

4. Testing

Evaluate performance and functionality

5. Deployment

Release and maintain the system



Clear Requirements Prevent Costly Rework and Misalignment

Requirements

Specific parameters the system must satisfy.
Defines what the system will do.

"System must fit in a pocket."

Specifications

Detailed parameters describing how the system works to meet requirements.

"Dimensions: 120mm x 60mm x 10mm."

Constraints

Limitations within which the system must operate, often forcing tradeoffs.

Cost, Safety, Compatibility, Schedule.

Key Measures

Safety

Accuracy

Precision

Response Time

Reliability

Strategic Design Decisions Determine System Architecture and Feasibility



High-Level Design

Builds a conceptual model of the hardware/software system. Focuses on information flow and abstraction while hiding implementation details.

Preliminary Design

Defines the top-down hierarchical structure, basic I/O signals, shared data structures, and the overall software scheme.

Low-Level Design

Specifies detailed hardware designs, mechanical drawings, and software module interconnections. Addresses safety and testing specifics.

Visualization Tools

Data Flow Graphs

Block diagrams showing the flow of information from source to destination.

Call Graphs

Graphical definition of how software and hardware modules interconnect.

Block Diagrams

High-level structural representation of system components.

Stepwise Decomposition Transforms Complex Problems into Manageable Code

The Method

1. Decompose

Break the main task into simpler subtasks.

2. Refine

Break subtasks into even simpler sub-subtasks.

3. Code

Continue until each unit is simple enough to implement directly.

4 Building Blocks

Sequence

"do A then do B"

Standard linear execution flow.

Conditional

"if A, then do B"

Decision making logic.

Iteration

"repeat A until B"

Loops (for, while) for repetitive tasks.

Interrupt

"on event, do B"

Crucial for Embedded: Hardware-triggered software functions.

Code Quality Determines Maintainability, Debuggability, and Long-Term Success



Quantitative Criteria

Dynamic Efficiency

Execution speed and responsiveness.

Static Efficiency

Memory requirements (RAM/ROM usage).

Accuracy

Difference between expected truth and actual results.

Qualitative Criteria

Maintainability

Ease of adding features or fixing bugs.

Readability

Clarity over cleverness; easy to understand.

Verifiability

Ease of proving correctness and testing.

The Golden Rule of Software Development

"Write software for others as you wish they would write for you."

Simulation and Prototyping Accelerate Development and Reduce Risk

01. Translation

Cross-Compilation

Converts source code into machine code for the target system, bridging the gap between development environment and embedded hardware.

02. Modeling

System Simulation

Models hardware and software behavior together, allowing verification of logic before physical components are available or built.

03. Iteration

Rapid Prototyping

Enables multiple fast loops around the design cycle (Analysis-Design-Implementation-Testing), leading to a more sophisticated final product.

04. Insight

Real-time Analysis

Permits the study of complex, time-critical interactions in a controlled environment where time can be paused or inspected.

Rigorous Testing Ensures Correctness, Performance, and Reliability

Testing is a progressive phase: first debug to validate basic functionality, then measure to optimize system performance.

01 **Static Efficiency**

Optimizing memory requirements (RAM/ROM usage) to fit within hardware constraints.

03 **Accuracy**

Minimizing the difference between the expected truth and the actual measured parameter.

02 **Dynamic Efficiency**

Maximizing execution speed and responsiveness to meet real-time deadlines.

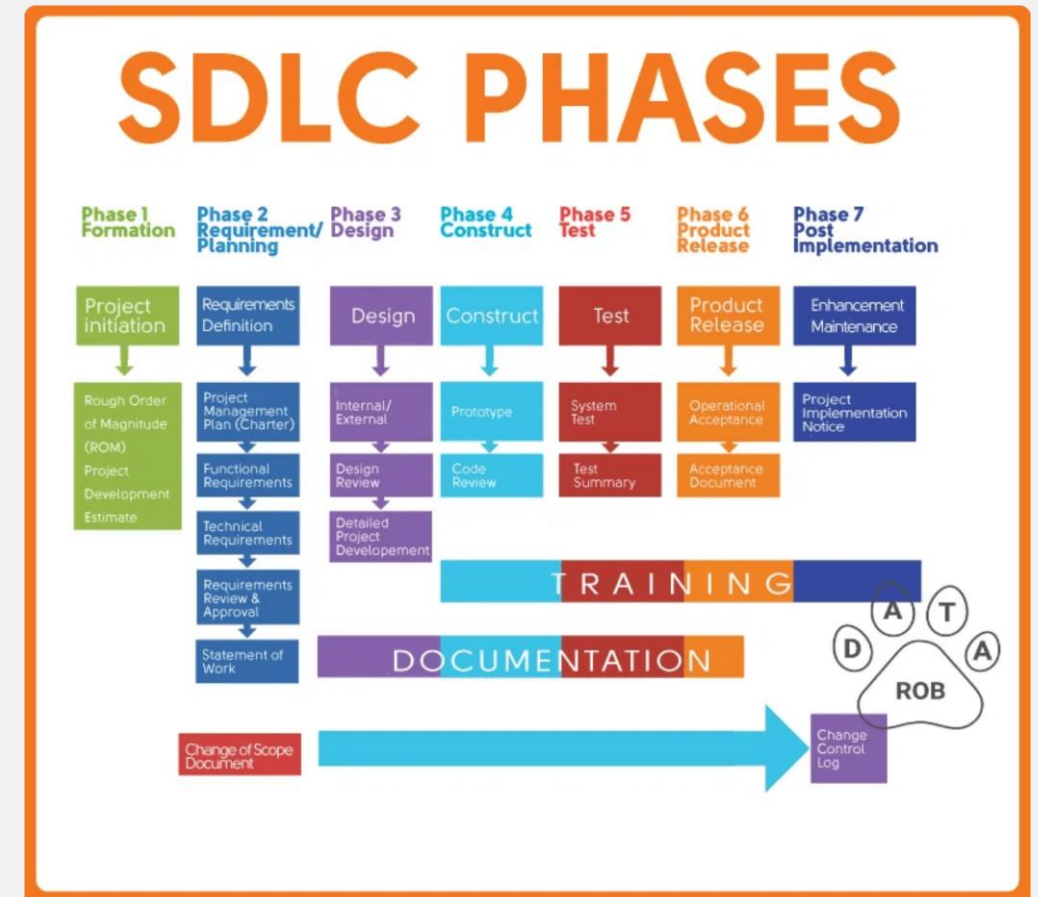
04 **Stability**

Ensuring consistent, reliable operation over time and under varying conditions.

Maintenance Extends System Life and Adapts to Changing Requirements

Maintenance is not a separate phase, but rather involves additional loops around the life cycle to ensure relevance and reliability.

- **Correcting Mistakes:** Fixing bugs and errors discovered post-deployment.
- **Adding Features:** Implementing new requirements from customers.
- **Optimizing:** Improving execution speed or reducing program size.
- **Porting:** Adapting the system to new computers or operating systems.
- **Reconfiguring:** Modifying the system to solve similar problems.



Choose Your Design Approach Based on Problem Understanding and Risk Tolerance

Top-Down



Problem → Solution

Starts with a clear problem statement and decomposes it into increasingly detailed specifications. Minimizes wasted effort on unused components.

Best Applied When

The problem is well-understood and the scope of potential solutions is clear.

Bottom-Up



Components → System

Begins with designing and testing low-level components, then fitting them together. Allows creative ideas to drive product development.

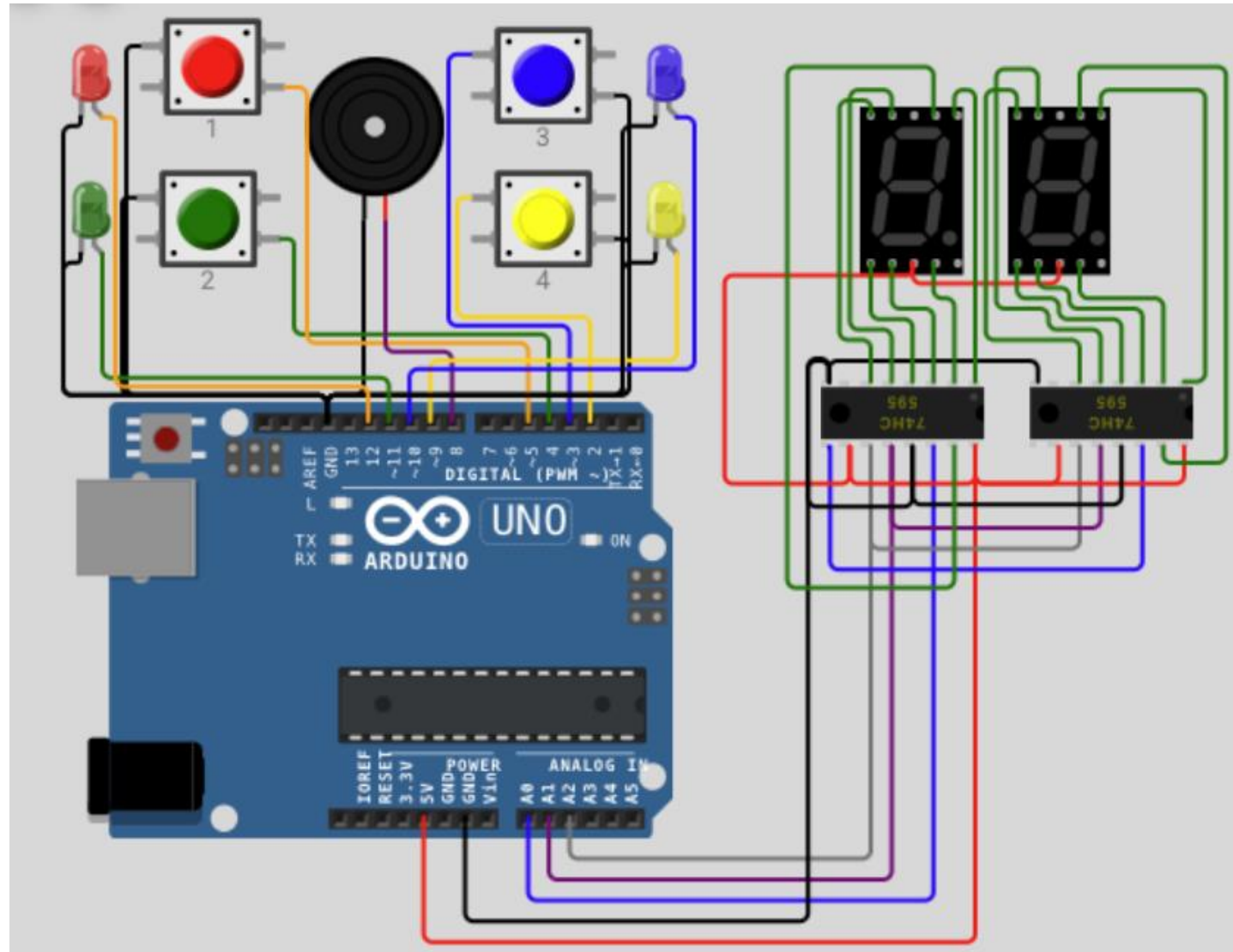
Best Applied When

Exploring new ideas, validating feasibility, or when the problem space is ambiguous.

Hybrid Approach

Most real-world projects combine both: Top-down for structure, Bottom-up for feasibility.

Virtual lab session recap: Simon game



Virtual lab session: Simon game

- Can you modify the rule so that the interval between flashes is halved?

Virtual lab session: Simon game

- Can you modify the rule so that the button must be pressed twice each time?

Thank you!